Asteroid Deflection Simulator
Design Documentation

Trevor Thomas

December 14, 2025

Contents

1 Introduction

The objective of this project was to develop a graphical user interface that was capable of rapidly exploring different test cases using an ion beam asteroid deflection simulator, leveraging MATLAB's computational swiftness and adaptability with built in dynamic numerical ODE solvers.

This project emphasizes more on workflow design. Its main purpose is to enable users to define, store, edit, and compare mission configurations without repeatedly editing source code to find desired outputs. A graphical interface was selected to reduce friction during iterative test runs of the MATLAB simulation and to centralize the input data to make each run easier to perform.

1.1 Background and Prior Work

The MATLAB simulation is what deals with all computational process in the backend of this project. The MATLAB simulation itself was developed previously for the AEROS Scholars program and now is part of my honors research. The simulation is an ion beam asteroid deflection simulation that utilizes MATLAB's ODE dynamic time step solvers. The sim follows continuous thrust from the ion beam thruster around an orbit of an asteroid for a specified angle window centered around the perihelion. The asteroid data is found using the NASA JPL Horizons data base, through web scraping, where intial condition r and v vector data can be obtained. The change in semi-major axis, eccentricity, and total B-plane deflection are then output to show the effects the ion beam thruster had on the asteroid. This is all in an effort to optimize thrust timing around an asteroids orbit to maximize the B-plane deflection relative to Earth. This simulation was validated against published planetary defense conference (PDC) papers by matching mission decisions and outputs.

While the MATLAB simulation proved to be fast with numerical computation, I found that it was not well suited for systematic trade studies with multiple changing inputs. Each test case required extensive manual modification of input parameters. As the number of scenarios increased, this workflow became both time consuming and error prone.

This GUI python project therefore focuses on building a structured graphical user interface around this existing MATLAB simulation, rather than changing code base or handling computation within python. By introducing a Python based GUI with persistent input profile management, the simulation becomes significantly more accessible while preserving the validated MATLAB simulation.

1.2 Design Philosophy

The design philosophy of this project is guided by three principles:

1. **Preserve validated simulation**: The MATLAB solver is treated as a trusted backend and remained unchanged throughout this project.

2. **Built-in Reference Case Protection**: Built-in profiles serve as protected reference cases, while custom profiles are created via duplicating and modifying reference cases rather than directly modifying them.

3. **Optimize iterative Test Runs**: The system is designed to minimize overhead between test runs and to fasten the process to run trade studies.

This philosophy reflects common practices in physics simulators where numerical computation is insulated from the user interface and the configuration data is protected from outside modification.

2 Design Process

Throughout early development, the initial goal was to directly integrate my MATLAB logic into Python and then developing a GUI around that. As development progressed, several technical issues began to surface. These issues were particularly related to time spent debugging and reproducing validated results. These challenges motivated multiple iterations of the overall design for this project before settling on the batch file execution of the untouched MATLAB simulation on the backend and the Python GUI on the front end.

2.1 Rejected Alternatives (In order)

2.1.1 Python-Only Numerical Propagation

One alternative was to reimplement the orbital propagation and ion beam dynamics entirely in Python using libraries such as SciPy. This approach was rejected for two primary reasons:

First, for this specific application, MATLAB offers superior performance and numerical stability for looping long duration ODE integration with several outputs. While Python is highly capable for many scientific tasks, the existing MATLAB implementation was already optimized and validated with reliable test runs of multi-year propagation windows.

Second, rewriting the simulation introduced unnecessary risk and a long time commitment. As previously stated, the MATLAB simulation had already been developed and validated as part of an AEROS Scholars project. Reproducing that level of confidence in a new language would require significant additional verification effort and time without providing any clear benefit.

2.1.2 MATLAB Engine API

The MATLAB Engine API for Python was evaluated as a means of directly invoking MATLAB functions from within the GUI. While functional, this approach was rejected due to stability and workflow concerns during the early prototyping stages of this project's development. Using the MATLAB engine library, the program had a tendency to crash unexpectedly due to inconsistency with environment variables and memory allocation between python and MATLAB. As these issues were hard to isolate and debug, the MATLAB engine was ultimately decided against for this project.

2.2 Chosen Method: MATLAB Batch Execution

MATLAB batch execution was selected as the primary simulation backend because it closely mirrors the desktop application while allowing the simulation to be driven using batch script execution on the backend from an external graphical interface. This approach enabled the already existing validated MATLAB simulation to be reused without modification, significantly reducing any risk of error and time spent validating code in development.

The batch execution demonstrated computational performance comparable to running the same scripts directly within the MATLAB desktop environment No measurable performance penalty was observed when invoking simulation through the batch mode, even for longer duration orbital propagation.

Matlab batch scripts are also straightforward to generate and modify which made developing a backend system for them very easy. It is very easy to print out statements during the script call. This made debugging much less tiring and easier to check if inputs were actually being handled correctly through the transition from python to MATLAB

3 High Level Overview of Project

On a higher level system view, this project separates into three distinct layers:

- **User Interface Layer**: A Python/PyQt GUI responsible for parameter input, profile management, and visualization.
- **Persistence Layer**: JSON based storage for asteroid, thruster, spacecraft, and scenario profiles.
- **Computational Backend**: A MATLAB batch execution that performs all orbital propagation and deflection calculations.

Each simulation run is fully defined by a generated script in python and produces a single `.mat` output file, which can be easily manipulated to display data trends within the GUI. This method enables straightforward visualization and comparison between scenario runs.

With the more modern Python GUI, this project achieves an increased user experience while keeping the swiftness and validated numerical computation of MATLAB preserved to enable efficient testing of ion beam asteroid deflection optimization.

4 Main GUI Module: `gui_main.py`

The core of the application resides in `gui_main.py`, which defines the `AsteroidSimGUI` class. This class inherits from `QWidget` and encapsulates all GUI elements, data loading, and simulation execution and output results.

4.1 Imports and Dependencies

The primary dependencies of `gui_main.py` are summarized below:

- `PyQt5`: Provides the graphical user interface framework, including widgets, layouts, dialogs, and the signal&slot event system
- `numpy`: Handles vector and array manipulation for simulation inputs and outputs returned from MATLAB
- `scipy.io`: Enables seamless loading and parsing of MATLAB
- `subprocess`: Launches MATLAB in batch mode and manages external process execution without blocking the GUI
- `matplotlib`: Generates visualization of orbital elements and deflection metrics in a series of plots
- Custom utility modules (`utils`, `utils_gui`, and `editor`): Provide JSON persistence, profile editing window dialogs, validation logic, and GUI helper functions

Main GUI Design Considerations:

- Numerical computation and orbital dynamics are intentionally excluded from the GUI layer to maintain responsiveness and avoid duplicated physics logic
- MATLAB batch execution via `subprocess` allows reuse of a validated simulation backend while still keeping the Python interface less computationally involved, making the GUI easier to extend and adapt to future use cases.
- Persistent JSON storage enables rapid configuration changes and repeatable simulation setups without manually editing source code
- Separation between GUI control, profile editors, and batch execution improves simplifies debugging and allows for easier modification for future needs

5 AsteroidSimGUI Class Initialization and Data Management

The knowledge of a constructor method was introduced in class and was utilized extensively in this project. Inheritance and object oriented program was more expanded upon with outside research. In this project, the AsteroidSimGUI class serves as the central composition root for the GUI. It owns all persistent data structures, JSON files, and application states. The editor and util modules reference this class as their authoritative source of data via parent ownership.

5.1 `__init__()`

The constructor initializes:

1. Initializes the Qt base class
2. Assigns JSON files to persistant storage filesnames for each profile category
3. Creates default non-editable built-in profiles to be loaded in with the program
4. Loads in and/or creates JSON file for asteroid, thruster, spacecraft, and scenario data
5. Sets built in protections for asteroid, thruster, and spacecraft data to prevent accidental modification of reference case configurations.
6. Runs the GUI constructor (initUI) to initialize all Qtwidgets and layout of the GUI

5.2 JSON-Based Data Persistence

Profiles and scenarios are stored using structured JSON files. This approach:

- Enables easy inspection and version control
- Separates GUI logic from data storage
- Allows flexibility in case future migration to databases is needed

6 Graphical User Interface Construction

6.1 `initUI()`

The `initUI()` constructor method is responsible for assembling the entire graphical user interface. This constructor method strictly defines widget creation, layout structure, and event handling.

The `initUI()` method constructs all widgets and layouts:

1. Creates all Qt widgets used for user interaction: including profile selectors, numerical input fields, action buttons, and status labels
2. Registers signal connections to bind user interactions (e.g. dropdown changes and button presses) to internal handler methods
3. Organizes widgets using a combination of `QFormLayout`, `QVBoxLayout`, and `QHBoxLayout`
4. Assigns `QVBoxLayout` as the layout manager for the main application window
5. Loads/refreshes asteroid, thruster, spacecraft, and scenario dropdowns from persistent JSON profile data
6. Loads the currently selected profiles immediately after refresh in order to initialize input fields with default values

UI Design Decisions:

- A `QFormLayout` was selected for numerical inputs in order to place labels directly adjacent to their corresponding inputs to reduce ambiguity in parameter identification
- All dropdowns are refreshed before loading selections to ensure the GUI state always reflects the current contents of JSON data

6.2 Signal&Slot Connections

Qt's signal and slot mechanisms were found through outside research. These mechanisms were used to implement an event driven GUI model, allowing the interface to respond immediately to user input.

- Scenario selection changes trigger automatic loading of saved scenario states into the GUI
- Edit buttons launch parameter editor dialog window pop ups for asteroid, thruster, and spacecraft profiles
- The run simulation button initiates the MATLAB batch pipeline using the current GUI state

The event driven approach of the GUI enables all interaction with the GUI to be handled in the backend. This significantly improves the user experience when running numerous test cases. It reduces the need for manual modifications and minimizes the likelihood of errors during each scenario setup.

7 Profile Editing and Protection Mechanism

The application supports three primary profile types:

- Asteroid
- Thruster
- Spacecraft

Each profile type has a set of built-in protected default profiles as well as user defined profiles stored in persistent JSON files. The built-in profiles serve as validated reference cases and cannot be overwritten. To enforce this protection editor dialogs for built-in profiles are launched with `editable=False` while user-created profiles can be modified freely.

7.1 Editors Module Overview

Profile editing is handled in a separate `editors.py` module. This design decision was made to clearly separate between the main GUI and profile modification. This reduces code complexity and makes it easier to find specific methods.

Each profile type has its own editor class:

- `AsteroidEditWindow`
- `ThrusterEditWindow`
- `SpacecraftEditWindow`

These editor classes are implemented as `QDialog` subclasses. They contain form based input fields for all relevant parameters(ie., mass, radius, thrust, Isp, diveregence angles, etc.). Signals from input widgets are connected to validation checking and save/delete actions.

Editors Design Considerations

- **Separation of Functions:** Moving profile editing into a separate module keeps the main GUI code focused on scenario setup, input handling, and simulation execution.
- **Form Layouts:** Editors module use `QFormLayout` to align labels and inputs clearly, reducing ambiguity and displaying physical units/descriptors alongside each parameter.
- **Editable Flag:** Editors accept an `editable` parameter to enforce protection of built-in profiles. When set to `False`, the Save and Delete buttons are disabled, but users can still "Save As New" to create new profiles deriving from reference cases.
- **Persistent Storage:** All changes are saved back to the respective JSON file. This ensures that any modification of inputs persists across application sessions.
- **Input Validation:** Each editor validates numeric input to prevent invalid or negative values from being stored to reduce errors in running simulation.

This modular and protected approach ensures that profile data integrity is maintained while still allowing flexibility for the user to create and modify custom profiles for simulation scenarios.

7.1.1 Asteroid Profile Editor

The `AsteroidEditWindow` manages asteroid physical and ephemeris parameters:

- Mass and radius
- JPL Horizons object ID
- Optional Horizons start/stop dates and step size

Built-in asteroids cannot be overwritten but can be cloned and modified using the "Save As New Profile" feature. Validation ensures positive values for mass and radius. Upon saving the profile, all changes are persisted to `asteroids.json` and the main GUI dropdown is refreshed. This ensures that any scenario run uses the most up-to-date asteroid parameters whether they are derived from a reference profile or a user modified clone.

7.1.2 Thruster Profile Editor

The `ThrusterEditWindow` handles electric thruster parameters:

- Thrust magnitude (N)
- Specific impulse (Isp, s)
- Beam divergence angle (deg)

Built-in thrusters are protected from modification but can be cloned as well. Input validation ensures physically meaningful values: thrust and Isp must be non-zero and positive, while divergence angles must be non-negative. This separation from spacecraft configurations allows future extensions, such as power draw (wattage), to be implemented in the future when the computational backend is developed for it.

7.1.3 Spacecraft Arrangement Editor

The `SpacecraftEditWindow` manages spacecraft architecture parameters:

- Number of thruster arrays
- Number of spacecraft
- Fuel mass per spacecraft
- Spacecraft dry mass

All inputs are validated to ensure integer spacecraft counts and physically reasonable mass values. By separating spacecraft arrangement from thruster parameters, the GUI allows users to explore optimizations for the number of spacecraft and thruster arrays independently from thruster configuration, which is critical for trade studies focused on mission feasibility.

7.1.4 Workflow Summary

1. User selects a profile in the main GUI dropdown.
2. Editor dialog window is launched, already populated with the selected profile's parameters.
3. User modifies parameters or clones/modifies the profile with new name.
4. Changes are validated and saved to JSON.
5. Main GUI dropdowns are refreshed to reflect the updated or new profile.

This workflow reduces errors when iteratively adjusting parameters for multiple test scenarios and improves the user experience by avoiding direct modifications to the backend source code for each test.

8 Persistent JSON Data Handling: `utils.py`

The `utils.py` module provides utility functions for managing persistent storage of profile and scenario data in JSON format. The knowledge of JSON file handling came mostly from outside research besides the python string editing that was taught in class. This design ensures that user-created and default profiles are maintained across application sessions.

8.1 Core Functions

- **Directory Management:** `ensure_data_dir()` creates a dedicated `./data/` directory folder if it does not exist. All JSON files reside in the data folder.
- **Path Resolution:** `json_path(filename)` returns the full path to a JSON file in the data directory.
- **Loading JSON:** `load_json(filename, default_data)` attempts to load a JSON file. If the file does not exist, it is created with the provided default data, ensuring a consistent starting state for new users.
- **Saving JSON:** `save_json(filename, data)` writes the provided dictionary to disk with indentation for human readability.

JSON Design Considerations

- **Automatic Initialization:** Default data is automatically saved if a JSON file is missing, preventing runtime errors.
- **Centralized Storage:** All JSON files are stored under `./data/` which simplifies backups and changes in the future.
- **Human-Readable Format:** Indented JSON allows users to inspect or manually modify files easily if necessary.

This module ensures that all profile and scenario data persists between sessions while providing a simple and reliable API for the main GUI and editor modules.

9 MATLAB Batch Execution Interface

The `run_matlab_batch` function provides a bridge between the Python GUI and the validated MATLAB simulation backend. By executing MATLAB in batch mode, the GUI remains separate from the validated computation of the MATLAB simulation. The knowledge of this came from outside research and knowledge from previous projects.

9.1 Function Overview

`run_matlab_batch` accepts all necessary simulation parameters, including:

- Asteroid properties (mass M_A and radius R_A)
- Initial asteroid state vectors ($\mathbf{r}_0$, $\mathbf{v}_0$)
- Thruster parameters (thrust, specific impulse, beam divergence angle)
- Mass properties (fuel and dry mass)
- Spacecraft arrangement (array number, number of spacecraft)
- Simulation step size (dt)
- Mission parameters (standoff distance, `orbit_window`, `infront`)

The function dynamically generates a MATLAB script, writes it to disk as `batch_script.m`, runs MATLAB via `subprocess`, and then retrieves results from a MATLAB `.mat` file for use in Python.

9.2 Data Flow and Processing

Main steps of the batch execution process:

1. **Python to MATLAB Conversion:** Python arrays (e.g., position and velocity vectors) are converted to MATLAB compatible syntax.
2. **Batch Script Generation:** The MATLAB script defines all variables explicitly and calls `ODE_Handle_GUI`, the primary numerical integrator. A `try/catch` block captures errors and ensures that they are written to `batch_output.mat` for debug purposes.
3. **MATLAB Execution:** MATLAB is invoked using the `-batch` flag. Standard output and error messages are captured for logging and debugging.
4. **Data Recovery:** On completion, Python loads `batch_output.mat` via `scipy.io.loadmat`. Arrays such as time history, position, velocity, orbital elements, B-plane deflection, and cumulative delta-v are extracted and flattened as needed.
5. **Error Handling:** MATLAB exceptions are detected and converted into Python `RuntimeError` objects which enables GUI error reporting without crashing the application.

MATLAB Batch Design Considerations:
Several design decisions were made to optimize robustness, usability, and performance:

- **Modularity:** MATLAB execution is fully decoupled from the GUI, allowing the Python interface to focus on user interaction and visualization while MATLAB handles numerical computation (also keeping validity of physics code in tact).
- **Explicit Variable Definitions:** All inputs are defined in the batch script to ensure deterministic behavior and reproducibility.
- **Error Containment:** The `try/catch` block in MATLAB ensures that runtime errors are captured and returned to Python without crashing the GUI.
- **Python-Friendly Output:** MATLAB outputs are saved in a structured `.mat` file and converted into NumPy arrays enabling visualization and data handling in Python.
- **Ease of Maintenance and Extension:** Batch scripts can be modified independently to add new features, such as power handling or multi-asteroid test cases without altering the main GUI code.
- **Performance:** Running MATLAB in batch mode achieves speeds comparable to direct desktop execution allowing rapid tests of multiple scenarios to be run.

9.3 Impact on Overall System Design

This architecture enforces a clear separation of responsibilities:

- Python GUI handles scenario setup, parameter editing, and visualization.
- Verified MATLAB backend handles all physics based calculations and integration.

The modular design simplifies the debugging process and makes future extensions within the MATLAB simulation easier as it will not affect the Python based GUI.

10 MATLAB Integration

10.1 Initial State Retrieval from JPL Horizons

Asteroid state vectors are obtained via the JPL Horizons database. The Python GUI passes Horizons parameters (object ID, start/stop dates, step size) to a MATLAB function which is executed in batch mode:

```
matlab -batch "rA0=get_horizons_rv(...); save('init_state.mat','rA0');"
```

The resulting state vectors are loaded into Python using `scipy.io.loadmat` and used as initial conditions for orbital propagation. This allows the simulator to start from accurate ephemerides while keeping the GUI code focused on the visual interface.

10.2 Orbital Propagation via `run_matlab_batch()`

The `run_matlab_batch()` function also generates a MATLAB script (`batch_script.m`) containing all simulation parameters:

- Initial asteroid position and velocity vectors
- Thruster parameters (thrust, specific impulse, divergence)
- Spacecraft configuration (number of spacecraft, thruster arrays, fuel and dry mass)
- Asteroid physical parameters (mass, radius)
- Mission parameters (time step, orbit window, approach direction)

The script calls the primary numerical integrator `ODE_Handle_GUI`, which performs:

- Ion beam thrust application
- Fuel mass depletion
- Orbital element calculations (semi-major axis, eccentricity)
- B-plane deflection and cumulative delta-v calculations

10.3 Execution and Data Handling

MATLAB is invoked using the `-batch` flag to run the MATLAB scrip without needing to open the desktop MATLAB app. The batch script is wrapped in a `try/catch` block so that any runtime errors are captured and saved to `batch_output.mat`. Python then:

- Loads output using `scipy.io.loadmat`
- Flattens arrays into NumPy compatible formats
- Returns a structured dictionary containing time histories, state vectors (r & v), orbital elements, B-plane deflection, and delta-v accumulation

Execution Design Considerations

- **Separation of Concerns:** MATLAB handles all numerical computation while Python manages GUI input/output.
- **Reproducibility:** Batch execution ensures deterministic runs independent of the MATLAB workspace.
- **Error Containment:** Runtime errors in MATLAB are captured and reported in Python which prevents any GUI crashes.
- **Precision and Reliability:** Python arrays are converted to MATLAB syntax to avoid numerical inaccuracies and/or errors.

11 Result Visualization

11.1 Summary Dialog

After a simulation completes, a non-modal (does not block use of other windows) Qt dialog window presents mission outcomes. These include:

- Total B-plane (zeta) deflection
- Cumulative delta-v on the asteroid imparted by the spacecraft
- Change in orbital period of the asteroid

The dialog window is intentionally made to not block other windows. This allows users to continue interacting with the main GUI while reviewing results. This design improves usability for trade studies where multiple simulations may be launched sequentially.

11.2 Time Plots

Orbital data is visualized using Matplotlib in a separate window. The plots typically include:

- Semi-major axis evolution versus time
- Eccentricity evolution versus time
- B-plane deflection metrics versus time

The figures are interactive with features such as zooming, panning, and saving figures. This means the user will be able to thoroughly inspect data and visualize what each test case results in. Data from the batch output is automatically converted into NumPy arrays to plot using Matplotlib.

Design Considerations

- **Interactive Plots:** Users can examine trends over time and inspect anomalies directly within the plots.
- **Automatic Data Handling:** Structured dictionaries returned from `run_matlab_batch()` simplify plotting and reduce risk of misaligned arrays and errors between scenarios.

For full visual pictures of the final GUI, editor windows, and results of test runs, refer to the Appendix A at the end of this paper.

12 Design Considerations and Extensibility

The GUI was designed to balance usability, maintainability, and numerical rigor. Key design principles include:

- **Modularity:** Each component (GUI, profile management, and MATLAB simulation) is separated to allow independent updates or replacement. This facilitates changes within either the simulation or GUI, depending on whatever is needed in the future.
- **Separation of UI and Computation:** The Python GUI is responsible only for input handling, visualization, and presenting a graphical interface, while MATLAB handles all numerical propagation. This ensures that the verified computations of the MATLAB simulation is persevered even with the introduction of the python GUI.
- **Data Protection:** Built-in profiles are non editable, and all user modifications are stored in persistent JSON files. This minimizes the risk of accidental data modification and protects validated reference cases.
- **Robust Error Handling:** Batch MATLAB execution is wrapped in try/catch blocks with errors captured and returned to the GUI for user notification without crashing the application.

12.1 Extensibility

The modular structure supports a wide range of potential enhancements:

- **Power-Constrained Thrusters:** Integration of energy or solar array models could allow more realistic modeling of ion beam thrust capabilities.
- **Automated Parameter Sweeps and Optimization:** Structured JSON inputs and the batch execution interface make it straightforward to run large scale studies.
- **Alternative Numerical Solvers:** The Python interface could be adapted to communicate with other solvers if needed without altering the GUI graphical logic/workflow.

13 Concepts Learned

While building this Python GUI project, I used and learned several programming and software engineering concepts:

- **JSON File Handling:** I learned how to save and load data using JSON, which allowed the application to store asteroid, spacecraft, and thruster profiles persistently across different sessions. The proper handling of data ensured that data would be consistent across program sates and reliable for use in multiple test runs.
- **Subprocess Batch Script Execution:** I learned how to call MATLAB scripts from Python using the `subprocess` module utilizing batch scripts. This allowed the program to run simulations automatically without opening MATLAB manually and to handle errors and outputs from the scripts.
- **Building GUIs with PyQt:** I learned how to create windows, dialogs, input forms, and buttons with PyQt. I also learned how to connect user actions, like clicking a button or changing a dropdown, to code that updates data or runs the simulation.

- **Keeping Code Organized:** I learned how to separate different parts of the program so that the GUI, the data storage, and the MATLAB code could work independently. This separation reduces interdependencies, simplifies debugging, and allows modifications in one component without unintended effects on the other parts.
- **Checking Inputs and Handling Errors:** I learned how to efficiently validate and check that users enter reasonable values. I also learned how to handle cases where something goes wrong in MATLAB or Python by handling errors so that the GUI would not crash, rather just show what exception occurred. This type of defensive programming was rather new to me and took some time getting used to when implementing it into the project.

These skills helped me connect a Python interface to a complex MATLAB simulation and make it easier for the user to run tests consistently without manual modification to source code.

14 Retrospective

There were several aspects of this project that I could or would have approached differently:

- **Editor Methods:** I would have built a more robust system for the editors instead of copying and modifying one from the other. This would make it easier to expand the code if additional profiles were needed in the future.
- **Implementing Input Validation Earlier:** I spent a long time debugging issues caused by invalid inputs that were not immediately obvious. This experience pushed me to implement a more defensive approach to input handling. Once the verification methods were in place, I discovered that vectors were being handled incorrectly before being sent to the MATLAB simulation, resulting in NaN values and other unexpected outputs.
- **Researching Python to MATLAB Integration More:** I would have explored alternative methods for Python to MATLAB integration earlier. A significant amount of debugging went into attempting to get the MATLAB engine library to work, which ultimately presented too many hurdles. In future projects, I will take the time to research alternatives and weigh their pros and cons before investing hours into a solution that may not be necessary.

If I had more time, I would do the following:

- **Implement Power Handling:** I have not yet implemented power handling for thrusters and spacecraft in the MATLAB simulation, so it could not be included in the GUI. Given more time, I would implement it and observe how the results differ before and after this feature.
- **Enhance Graphics and Colors:** I am still relatively new to PyQt and aware of its complex capabilities. With more time, I would explore these features more thoroughly and potentially add enhanced graphics and color to the GUI.
- **Batch Automation:** Automating a series of tests using batch execution could significantly reduce user workload. With more time, I would implement a mode in the GUI that allows selecting independent variables for a series of tests, running them sequentially or simultaneously (likely requiring multi-threading).

15 Conclusion

I learned a lot from this project and successfully developed a GUI front end for my honors research project. This tool will facilitate future iterative test runs and simplify the modification of parameters between tests. I gained skills in data management, code organization, and UI building. I also gained experience running batch scripts and applying defensive programming techniques from the front end to the back end of the application.

References

1. JPL Horizons System. NASA Jet Propulsion Laboratory. https://ssd.jpl.nasa.gov/horizons/
2. PyQt5 Documentation. https://www.riverbankcomputing.com/static/Docs/PyQt5/
3. MATLAB Command-Line Interface: https://www.mathworks.com/help/matlab/matlab_external/run-matlab-functions-from-the-command-line.html
4. MATLAB Documentation: `-batch` mode. https://www.mathworks.com/help/matlab/ref/matlab-batch.html
5. MATLAB Engine API for Python Documentation. https://www.mathworks.com/help/matlab/matlab_external/get-started-with-matlab-engine-for-python.html
6. Python Standard Library Documentation: `subprocess`, `json`, `os`. https://docs.python.org/3/library/
7. SciPy Documentation: `scipy.io.loadmat`. https://docs.scipy.org/doc/scipy/reference/generated/scipy.io.loadmat.html
8. NumPy Documentation. https://numpy.org/doc/stable/

A Main Graphical User Interface

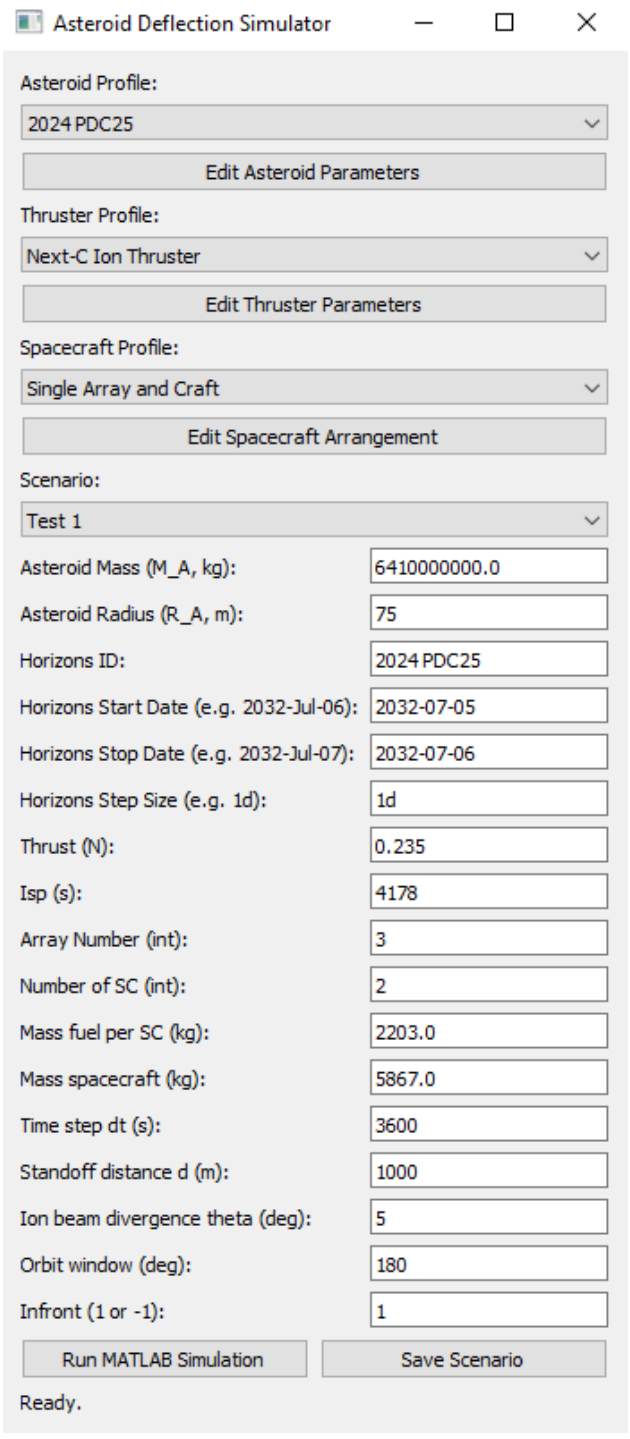

Figure 1: Main GUI for Asteroid Deflection Simulation

B Editor Windows

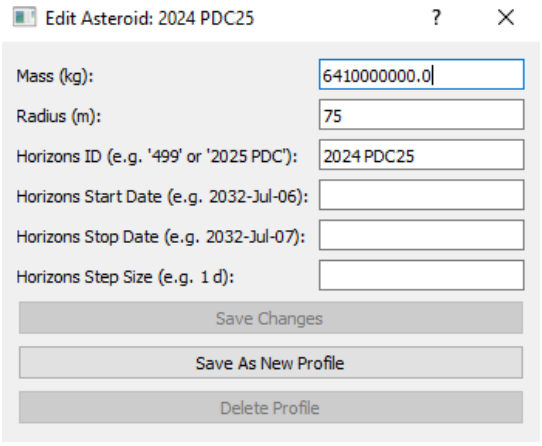

Figure 2: Asteroid Editor

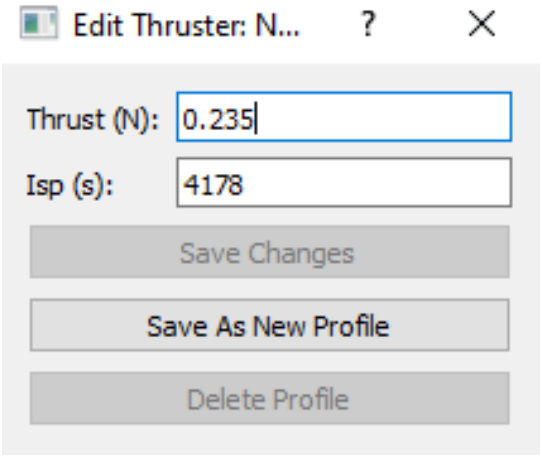

Figure 3: Thruster Editor

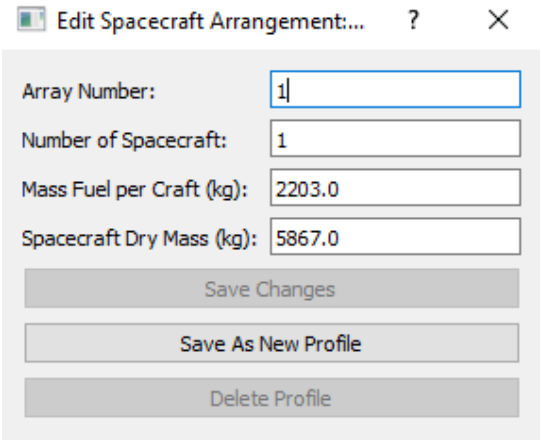

Figure 4: Spacecraft Editor

C Summary From Successful Test Case

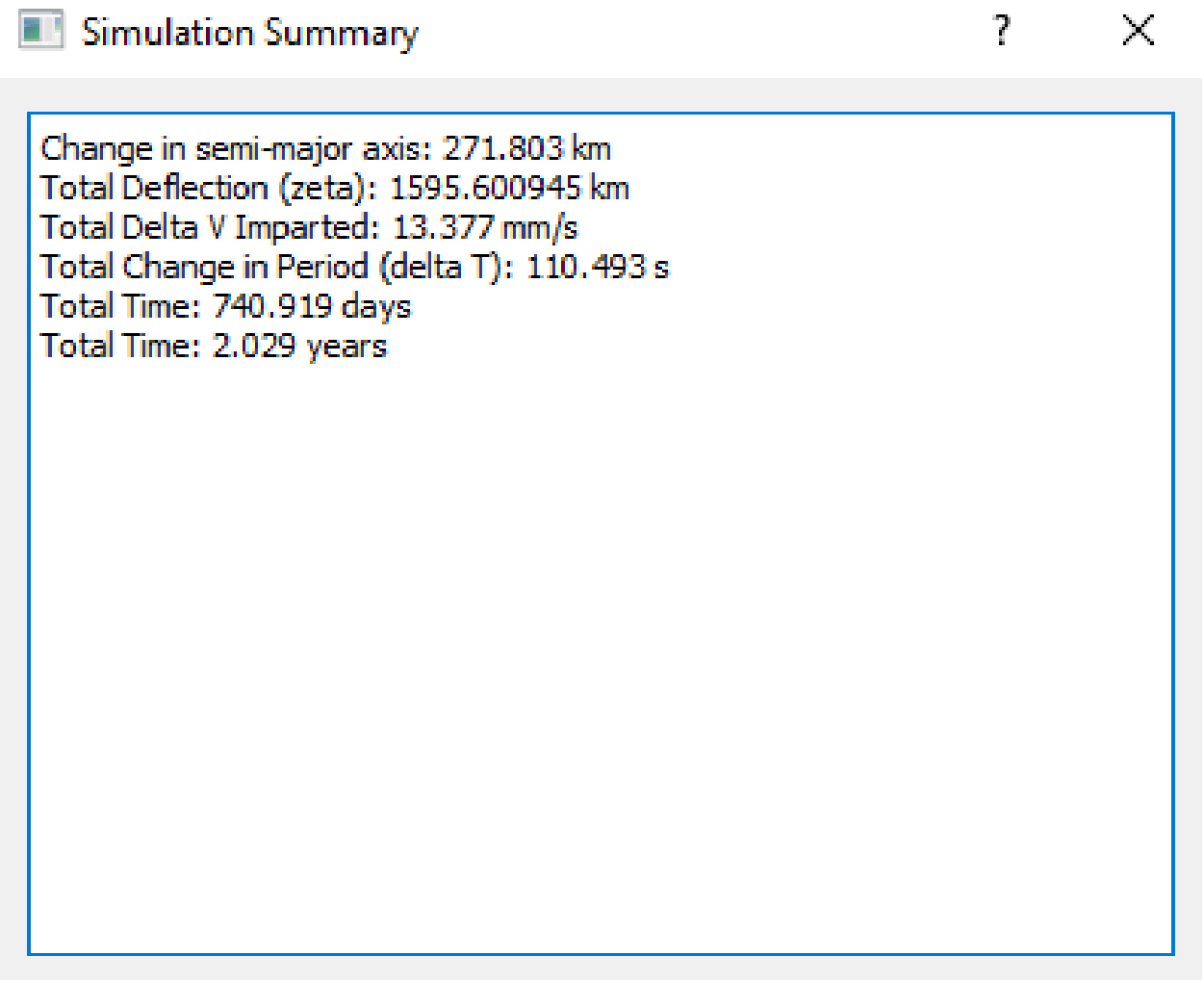

Figure 5: Simulation Summary

D Plots from Successful Test Case

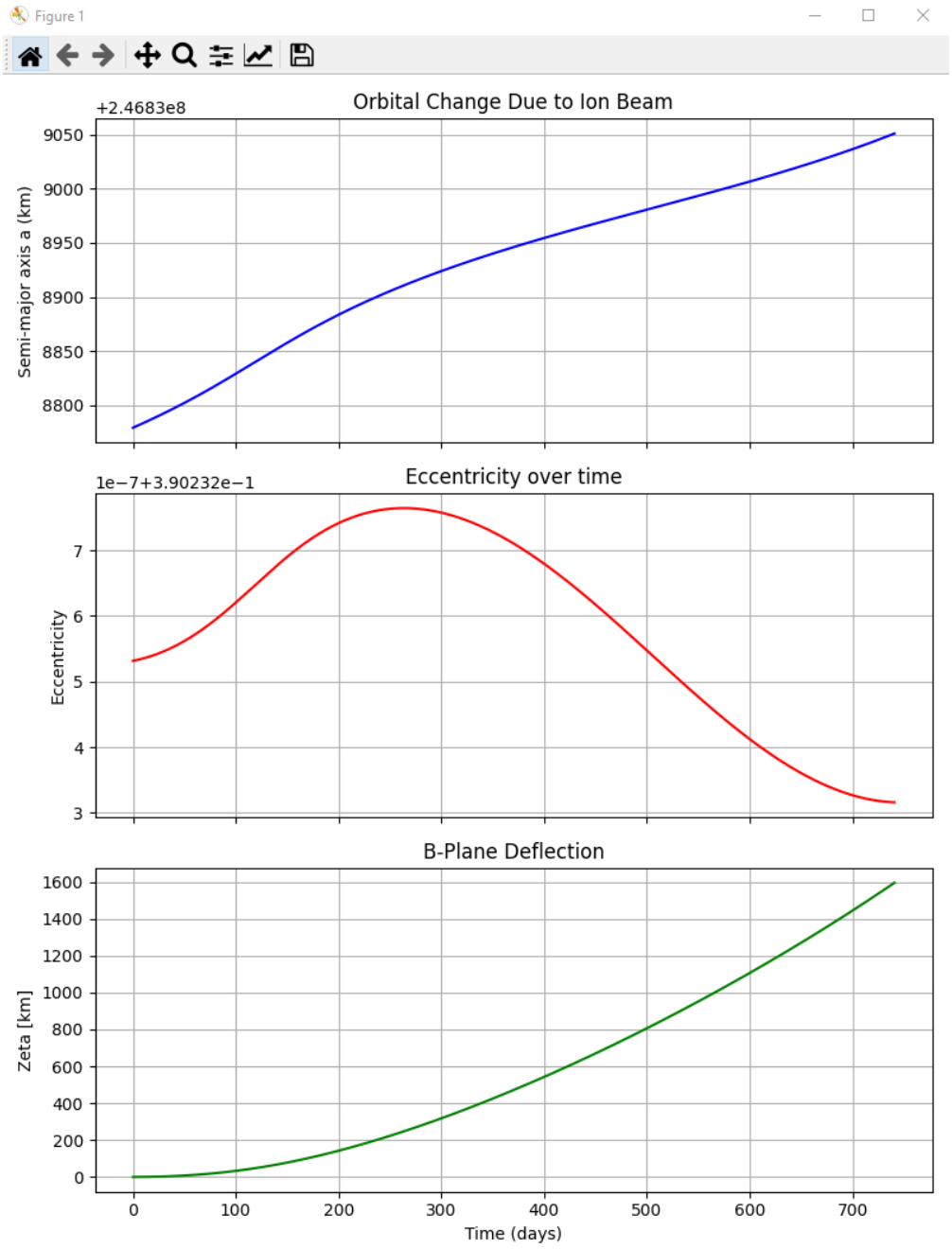

Figure 6: Subplots showing orbital change over time